

Prompts in LangChain

1 Introduction & Recap

This chapter focuses on **prompts in LangChain**, emphasizing how structured input improves LLM performance and consistency. It builds on earlier concepts by refining how we communicate with models.

Example: Instead of asking randomly, you structure input like: “Summarize this text in 3 bullet points.”

2 Error Correction

Prompt design helps reduce incorrect or vague responses by giving clear instructions and constraints. Well-defined prompts guide the model toward accurate outputs.

Example:

Bad: “Explain AI”

Better: “Explain AI in 3 simple points for beginners.”

3 Prompts

Prompts are the instructions given to the model to generate output. In LangChain, prompts are managed systematically to ensure reusability and consistency.

Example:

Prompt: “Translate ‘Hello’ to French” → Output: “Bonjour”

4 Static vs Dynamic Prompts

- **Static prompts** are fixed and do not change.
- **Dynamic prompts** include variables that are filled at runtime.

Dynamic prompts make applications flexible and user-driven.

Example:

Static: “Summarize this article”

Dynamic: “Summarize this article: {article_text}”

5 Prompt Templates

Prompt templates allow you to define reusable prompt structures with placeholders. This avoids rewriting prompts and ensures consistent formatting.

Example:

Template: "Answer based on context: {context}, Question: {question}"

→ Automatically filled during execution.

6 Messages

Messages structure conversations into roles like system, user, and assistant. This helps maintain context and control behavior in chat-based models.

Example:

System: "You are a helpful tutor"

User: "Explain gravity"

→ Model responds accordingly.

7 Chat Prompt Templates

These are advanced templates designed for chat-based models, combining multiple message roles into a structured format.

They enable multi-turn conversations with controlled behavior.

Example:

System + User messages combined → Model generates more context-aware responses.

8 Message Placeholder

Message placeholders allow dynamic insertion of past conversation or additional messages into prompts. This is useful for maintaining conversation flow.

Example:

Placeholder: {chat_history}

→ Inserts previous messages so the model remembers context.
